Experimental Instructions 2

In this experiment, we will learn how to deploy and use the Serverless open source platform OpenFaas.

• Step One –Set up the OpenFaaS environment

Create OpenFaaS Function

1. Enter the OpenFaaS workspace

```
1. cd /root/cloudz/functions
```

We have installed OpenFaaS in advance in the virtual machine environment.

2. OpenFaaS login

```
1. export OPENFAAS_URL=http://127.0.0.1:31112
2. faas-cli login --password admin
```

3. Create a Python function from template

```
1. faas-cli new --lang python3-flask hello-python
```

python3-flask is the **template** and hello-python is the **function name**. More templates can be found in the templates directory.

The command above will generate **hello-python.yml** and directory hello-python. hello-python.yml is used to build and deploy the function.

```
root@ubuntu:~/cloudz/functions# ls hello-python
handler.py  __init__.py  requirements.txt
root@ubuntu:~/cloudz/functions#
```

handler.py contains the source code of the function, and **requirement.txt** is used to import the third-party libraries.

4. Modify hello-python/handler.py

```
def handle(req):
    """handle a request to the function
    Args:
        req (str): request body
    """

    return "hello, " + req
```

5. Build function

1. faas-cli build -f ./hello-python.yml

```
root@ubuntu:~/cloudz/functions# faas-cli build -f ./hello-python.yml
[0] > Building hello-python.
Clearing temporary build folder: ./build/hello-python/
Preparing: ./hello-python/ build/hello-python/function
Building: hello-python:latest with python3-flask template. Please wait..
Sending build context to Docker daemon  9.728kB
Step 1/32 : FROM openfaas/of-watchdog:0.7.7 as watchdog
```

6. Deploy function

1. faas-cli deploy -f ./hello-python.yml

```
root@ubuntu:~/cloudz/functions# faas-cli deploy -f ./hello-python.yml
Deploying: hello-python.
WARNING! Communication is not secure, please consider using HTTPS. Letsencrypt.org offers f
LS certificates.

Deployed. 202 Accepted.
URL: http://127.0.0.1:31112/function/hello-python.openfaas-fn
```

7. List the functions have been deployed

1. faas-cli list

```
root@ubuntu:~/cloudz/functions# faas-cli list
Function                        Invocations     Replicas
hello-python                    0               1
```

List the details of a function

1. faas-cli describe hello-python

```
root@ubuntu:~/cloudz/functions# faas-cli describe hello-python
Name:                hello-python
Status:              Ready
Replicas:            1
Available replicas:  1
Invocations:         0
Image:               hello-python:latest
Function process:    python index.py
URL:                 http://127.0.0.1:31112/function/hello-python
Async URL:           http://127.0.0.1:31112/async-function/hello-python
Labels:              faas_function : hello-python
Annotations:         prometheus.io.scrape : false
```

8. Invoke OpenFaaS function

- faas-cli

```
echo Tom | faas-cli invoke hello-python
```

```
root@ubuntu:~/cloudz/functions# echo Tom | faas-cli invoke hello-python
hello, Tom
```

- curl

```
curl http://127.0.0.1:31112/function/hello-python -d "Tom"
```

```
root@ubuntu:~/cloudz/functions# curl http://127.0.0.1:31112/function/hello-python -d "Tom"
hello, Tomroot@ubuntu:~/cloudz/functions#
```

List the functions again, and you can find the invocations have been updated

```
root@ubuntu:~/cloudz/functions# faas-cli list
Function                      Invocations     Replicas
hello-python                  2               1
```

Step Two

Run OpenFaaS Function & Collect runtime information

To finish this exercise, you need to **run the following function**, and **collect the runtime information**, such as latency (cold start time & execution time), number of replicas, QPS(Query Per Second), resource consumption information of running replicas.

Sample Function 1: float-operation

```
faas-cli new --lang python3-flask float-operation
```

Modify float-operation /handler.py

```
import json
import math
from time import time

def float_operation(N):
    start = time()
    for i in range(0, N):
        sin_i = math.sin(i)
        cos_i = math.cos(i)
        sqrt_i = math.sqrt(i)
    latency = time() - start
    return latency

def handle(req):
    """handle a request to the function
    Args:
        req (str): request body
    """
    json_req = json.loads(req)
    n = int(json_req['n'])

    latency = float_operation(n)
    return "latency : " + str(latency)
```

Build & Deploy

```
faas-cli build -f ./float-operation.yml
faas-cli deploy -f ./float-operation.yml
```

Invoke

```
curl -d '{"n":100}' http://127.0.0.1:31112/function/float-operation
```

```
root@ubuntu:~/cloudz/functions# curl -d '{"n":100}' http://127.0.0.1:31112/function/float-operation
latency : 0.0008709430694580078root@ubuntu:~/cloudz/functions#
```

The result is the **latency** (second). And you **can replace 100 with other number** for different workload.

Sample Function 2: matmul

```
faas-cli new --lang python3-flask matmul
```

Modify matmul/handler.py

```
from time import time
import json
import random

def generate_random_matrix(n):
    """Generates an n x n matrix with random floats between 0 and 1."""
    return [[random.random() for _ in range(n)] for _ in range(n)]

def matmul(A, B):
    """Multiplies two n x n matrices A and B."""
    n = len(A)
    # Initialize the result matrix C with zeros
    C = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def calculate_latency(n):
    A = generate_random_matrix(n)
    B = generate_random_matrix(n)

    start = time()
    C = matmul(A, B) # Perform matrix multiplication
    latency = time() - start
    return "latency : " + str(latency)

def handle(req):
    """Handle a request to the function.

    Args:
    req (str): Request body as a JSON string.
    """
    json_req = json.loads(req)
    n = int(json_req['n'])
    result = calculate_latency(n)
    return result
```

Build & Deploy

```
faas-cli build -f ./matmul.yml
faas-cli deploy -f ./matmul.yml
```

Invoke

```
curl -d '{"n":100}' http://127.0.0.1:31112/function/matmul
```

The result is the **latency** (second). And you **can replace 100 with other number** for different workload.

Auto-scaling in OpenFaaS allows a function to scale up or down depending on

demand represented by different metrics.

The **minimum** (initial) and **maximum** replica count can be set at deployment time by adding a label to the function.

- **com.openfaas.scale.min** - by default this is set to 1, which is also the lowest value and unrelated to scale-to-zero
- **com.openfaas.scale.max** - the current default value is 20 for 20 replicas
- **com.openfaas.scale.zero** - set to true for scaling to zero

Example of modify hello-python.yml

```
version: 1.0
provider:
  name: openfaas
  gateway: http://127.0.0.1:31112
functions:
  hello-python:
    lang: python3-flask
    handler: ./hello-python
    image: hello-python:latest
    labels:
      com.openfaas.scale.max: 1
      com.openfaas.scale.zero: true
```

Deploy function again to update the configuration

```
1. faas-cli deploy -f ./hello-python.yml
```

Now the replicas count of hello-python will be 0 if no request for a while

```
root@ubuntu:~/cloudz/functions# faas-cli list
Function                        Invocations     Replicas
hello-python                    2               0
```

Replicas is actually docker container. For example, you can find hello-python replicas

```
1. docker ps | grep hello-python_
```

```
root@ubuntu:~/cloudz/functions# docker ps | grep hello-python_
04c6d17dd04b   2750e0f3b567                "fwatchdog"              10 minutes ago   Up 10 minutes
          k8s_hello-python_hello-python-667fdd4568-jmbr2_openfaas-fn_fd64cfcf-a760-461d-99d8-864ad177c68_0
root@ubuntu:~/cloudz/functions# _
```

Then, you can check docker resource

```
1. docker stats 04c6d17dd04b
```

```
CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O   BLOCK I/O     PIDS
04c6d17dd04b   k8s_hello-python_hello-python-667fdd4568-jmbr2_openfaas-fn_fd64cfcf-a760-461d-99d8-864aed177c68_0   0.03%     21.33MiB / 1.953GiB   1.07%     0B / 0B   17.6MB / 0B   13
```

Notice: id and name of replicas are not const

Command to update the number of replicas of a function directly

```
curl http://127.0.0.1:31112/system/scale-function/matmul -
d '{"ServiceName": "matmul", "Replicas": 0}' -u admin:admin
```

You can replace matmul and 0 with your function and specific numbers.

Use wrk to evaluate functions

Install wrk

```
apt install wrk
```

Create **post.lua** script with following content

```
wrk.method = "POST"
wrk.body = '{"n":100}'
wrk.headers["Content-Type"] = "application/json"
```

You can replace 100 with specific numbers as your wish.

Evaluate matmul function with **wrk** command

```
wrk -t 1 -c 1 -d 5s --latency -s post.lua http://127.0.0.1:31112/function/matmul
```

```
root@ubuntu:~/cloudz/functions# wrk -t 1 -c 1 -d 5s --latency -s post.lua http://127.0.0.1:31112/function
/matmul
Running 5s test @ http://127.0.0.1:31112/function/matmul
  1 threads and 1 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency    14.22ms   45.54ms 335.66ms   95.13%
    Req/Sec   232.89     69.37   380.00     70.21%
  Latency Distribution
     50%    3.18ms
     75%    6.76ms
     90%   10.20ms
     99%  280.83ms
  1106 requests in 5.05s, 210.33KB read
Requests/sec:    218.82
Transfer/sec:     41.61KB
```

wrk options

```
Usage: wrk <options> <url>
  Options:
    -c, --connections <N>  Connections to keep open
    -d, --duration    <T>  Duration of test
    -t, --threads     <N>  Number of threads to use

    -s, --script      <S>  Load Lua script file
    -H, --header      <H>  Add header to request
        --latency          Print latency statistics
        --timeout     <T>  Socket/request timeout
    -v, --version          Print version details
```

You can get the latency distribution of matmul function and the QPS (Req/Sec). You can use different options to evaluate the performance of functions.

Analyze these information and show your findings (e.g., text, tables, figures). You can also run any programs and runtimes (python, js, go, java,...) you want.

• Step Three

Update resource configuration

We can limit the resource that can be used by a replica. For example, constrain hello-function to only use 40Mb of RAM and 1/10 core at a maximum.

```
version: 1.0
provider:
  name: openfaas
  gateway: http://127.0.0.1:31112
functions:
  hello-python:
    lang: python3-flask
    handler: ./hello-python
    image: hello-python:latest
    labels:
      com.openfaas.scale.max: 1
      com.openfaas.scale.zero: true
    limits:
      cpu: 100m
      memory: 40Mi
    requests:
      cpu: 100m
      memory: 20Mi
```

- Requests ensures the stated host resource is available for the container to use
- Limits specify the maximum amount of host resources that a container can consume

You can check the docker stats after deploy the function again

```
CONTAINER ID   NAME
               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O     PIDS
da246bca7959   k8s_hello-python_hello-python-5fc56956c4-2rpjs_openfaas-fn_35b7b76a-d4d5-4fe0-93f8-c3
5db7948c69_0   0.01%     18.86MiB / 40MiB    47.14%    0B / 0B   1.54MB / 0B   12
```

Analyze changes of applications

Re-run your functions with limit resource. Record the changes under various restrictions. And compare the information of different functions.

Note: Show the analysis and result in your report. **The deadline is December 22nd!**